

ITC Assignment - Turing Machine

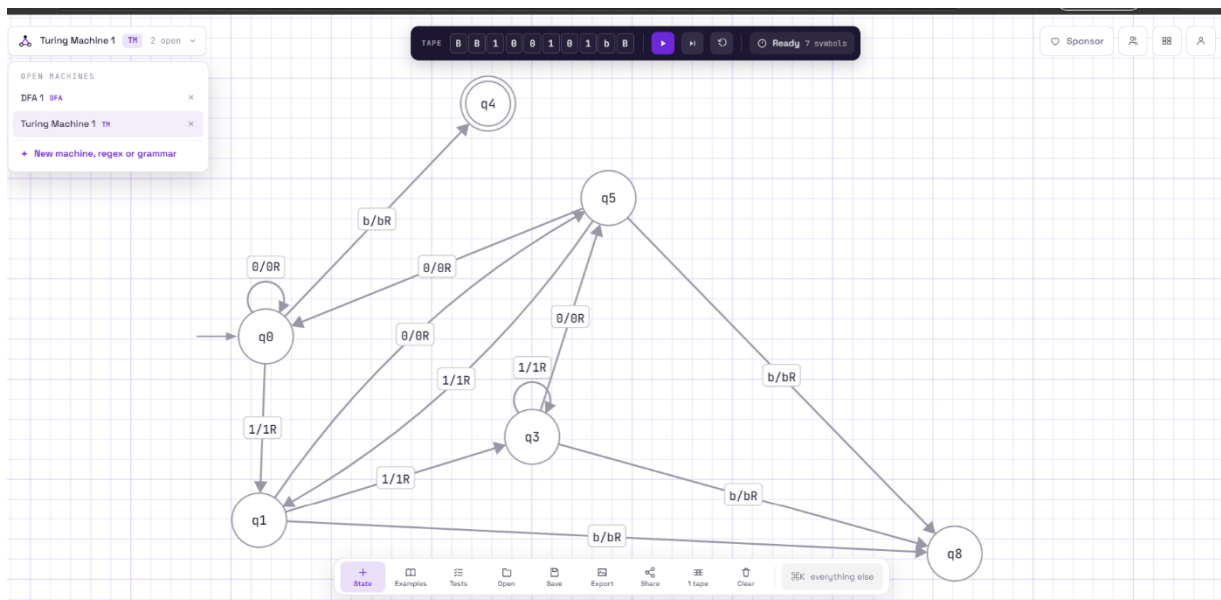
Personalization: R = 73, V = 1, B= 1001001, n=3

Task1 – A:

M=4

The logic Turing machine uses is that it uses modulus division and tracks the remainder and shifts state according to 2nd base left shifting. Basically, the machine reads the tape and figures out the remainder, changes state and moves to next bit. The formula looks like this: $(2 * r(\text{old}) + 1) \% 4$. Using this formula machine precalculated states from q0, q1, q5, q3, without processing the complete decimal number at a time. This is done using abstraction like the machine breaks down the process into smaller problems and figures out the result for each process. It halts upon encountering b at q4 and q8 is the reject state.

Task 1 – B:



q0, q1, q5, q3 – these are the transition states.

q8 is the rejection state.

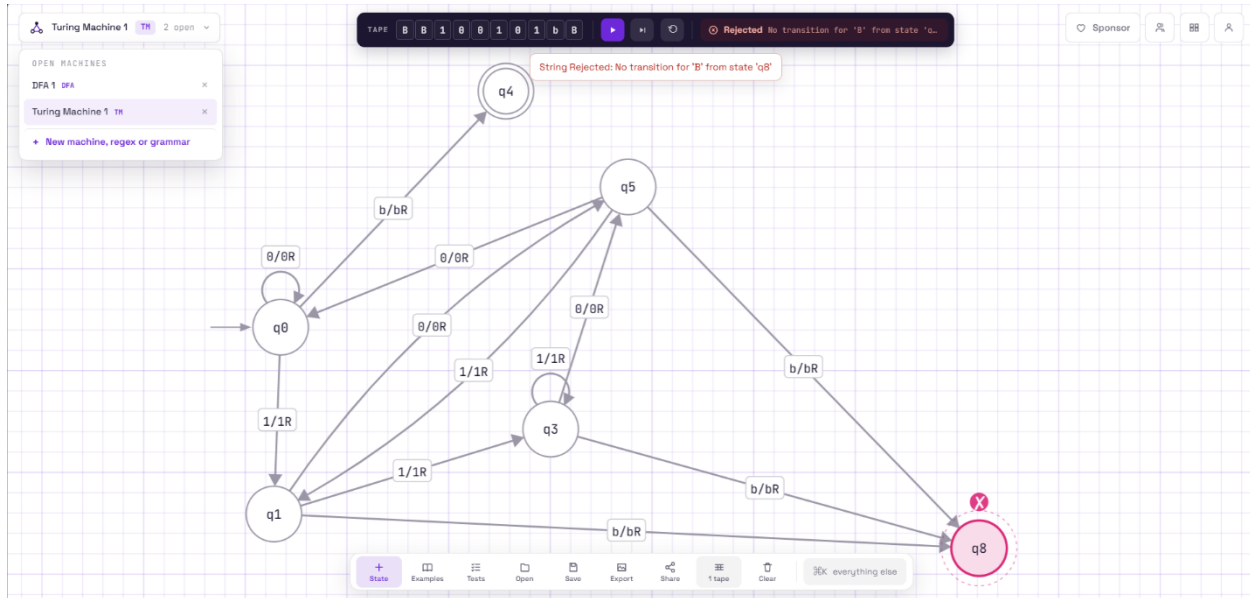
q4 is the acceptance state.

Task 1 – C:

Input Tape: 1001001 (Decimal 73)

Expected Result: Halts in q8 (REJECTED)

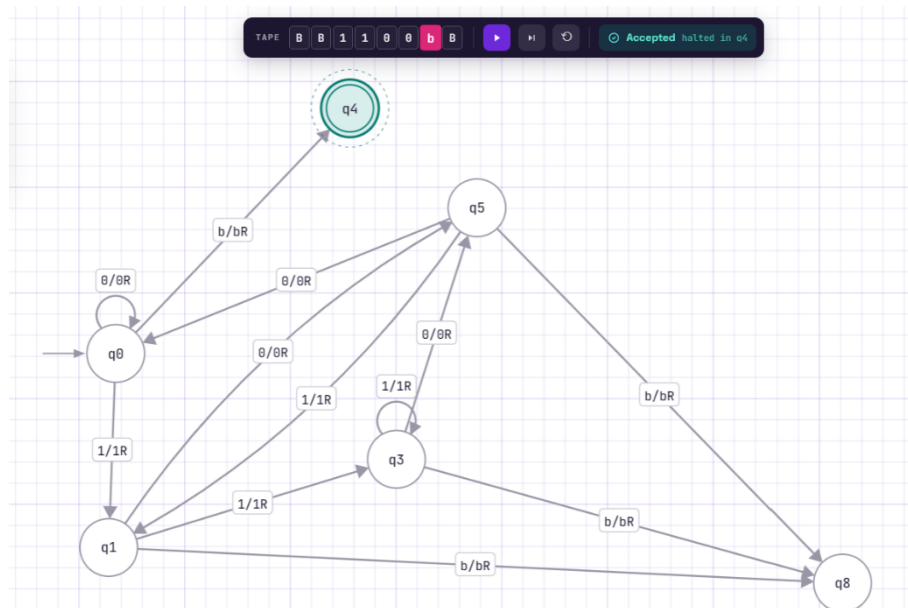
Path: Task1\png\TM_diagram_rejectstates#01.png



Input Tape: 1100 (Decimal 12)

Expected Result: Halts in q4 (ACCEPTED)

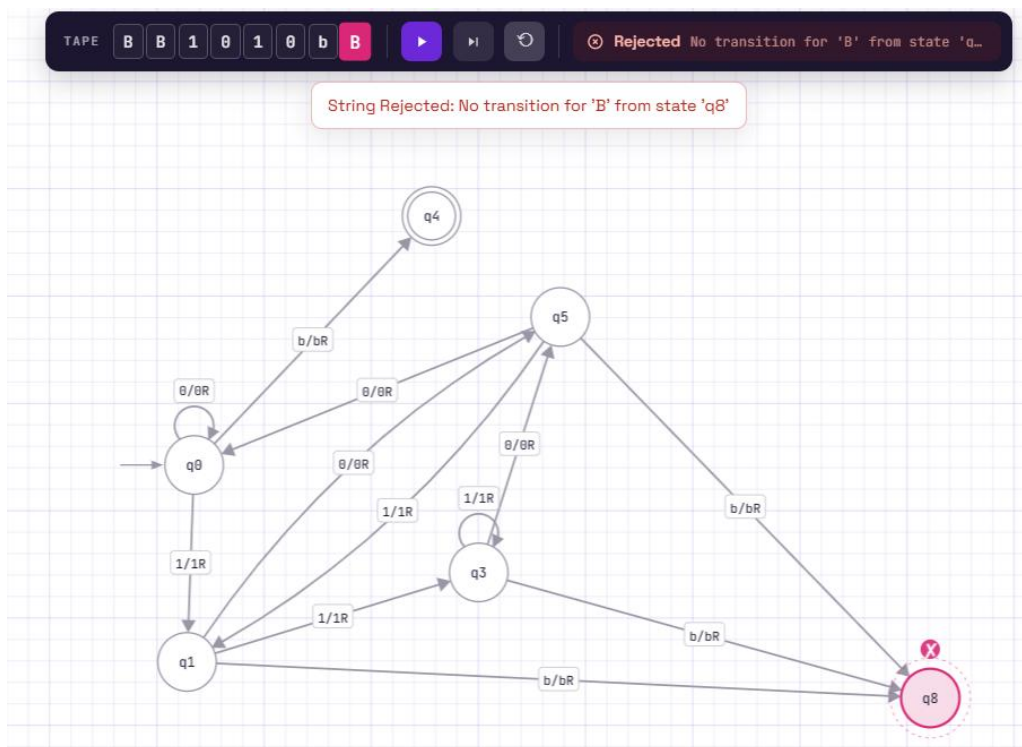
Location: \Task1\png\TM_diagram_acceptstate#01.png



Input Tape: 1010 (Decimal 10)

Expected Result: Halts in q8 (REJECTED)

Location: Task1\png\TM_diagram_reject#02.png



Task 1 - D:

M represents the transition states and remainders. In my case, $M=4$ so the formula I used required 4 states for 4 remainder. Plus every arrow and condition will be recalculated if the value of M changes.

Task2

Master Variant: $a^n b^{2n}$

N = 3

Task 2 - A: This machine checks the valid predetermined length of a string for an arbitrary value of n . The principles this turing machine works on is – it checks the first value of a writes x . it moves right of the tape and reads b , write Y . finds another b writes y and the tapes moves to the left finds x and then move right of the tape again to read a to write x , repeats the same.

The machine accepts the input, when the machines read y till it encounters B (halt/acceptance).

Task 2 - B:

(q0, 'a') -> ('B', R, q1)

(q0, 'B') -> ('B', R, q_accept)

(q1, 'a') -> ('a', R, q1)

(q1, 'b') -> ('b', R, q1)

(q1, 'B') -> ('B', L, q2)

(q2, 'b') -> ('B', L, q3)

(q3, 'b') -> ('B', L, q4)

(q4, 'a') -> ('a', L, q4)

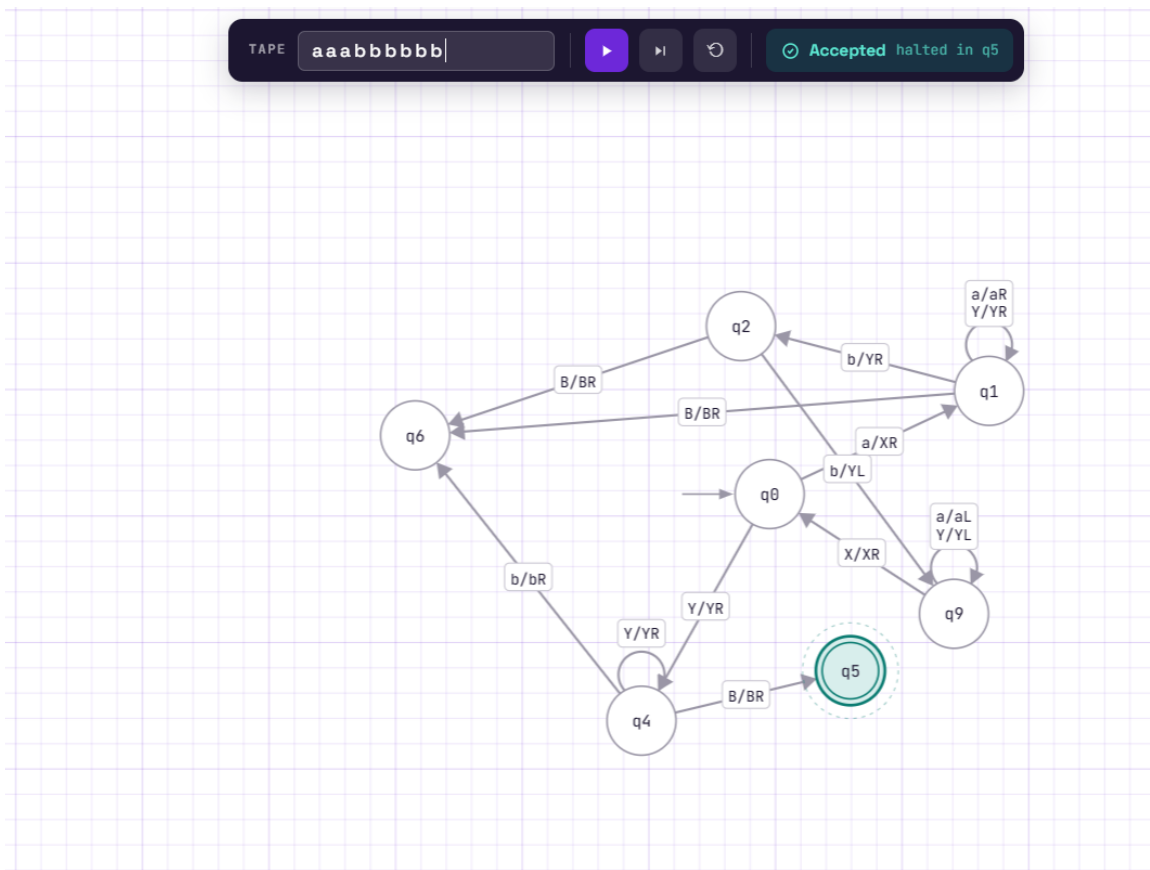
(q4, 'b') -> ('b', L, q4)

(q4, 'B') -> ('B', R, q0)

Task 2 - C:

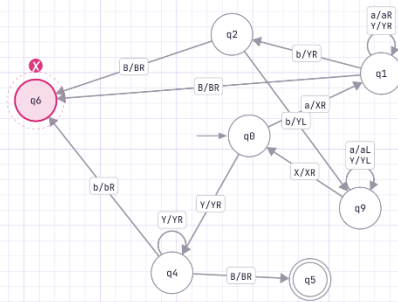
Test Case (i): $n = 3$

- Input String: aaabbbbbbb
- Expected Result: Accepted
- Actual Result: ACCEPTED (Halts in accept state q5)
- Evidence Files: Task2/q2_run1_aaabbbbbbb.json, q2_aaabbbbbbb_accept.png



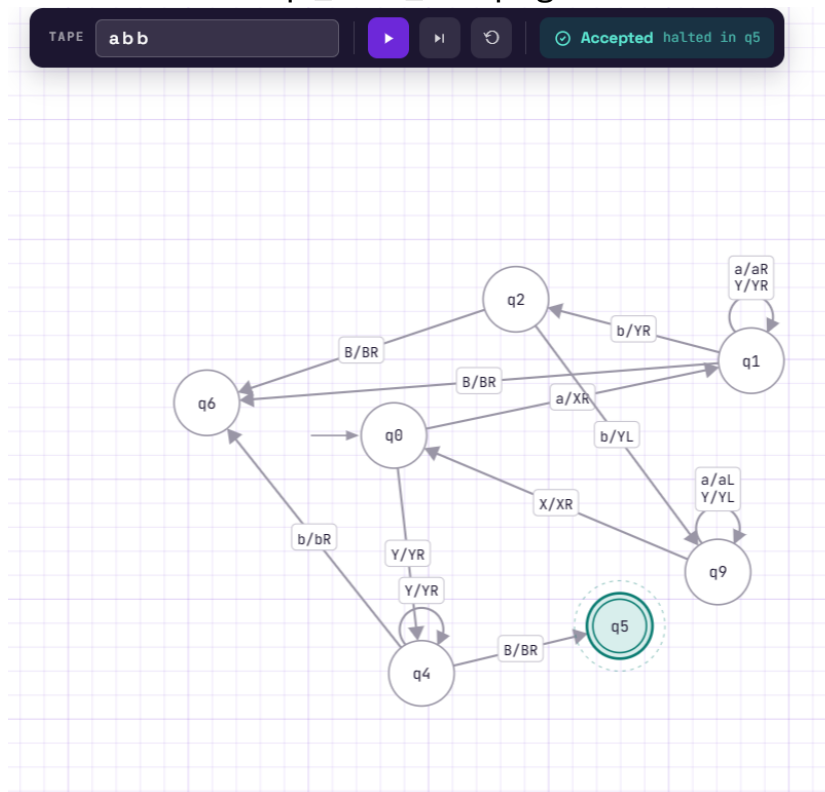
Test Case (ii) — Off-by-One String (1 Symbol Added)

- Input String: aaabbbbbbb
- Expected Result: Rejected
- Actual Result: REJECTED (Halts in reject state q6)
- Evidence Files: Task2/q2_run2_aaabbbbbbb_reject.png



3. Test Case (iii) — Custom Choice String ($n = 1$)

- Input String: abb
- Expected Result: Accepted
- Actual Result: ACCEPTED (Halts in accept state q5)
- Evidence Files: Task2/q2_run3_abb.png



Task 2 - D: my machine's design could be off by one error, if for the formula $a^n b^{2n}$ there may be 1 a to 3 b's mapped. Or failing to verify that all b's are consumed when we right sweep across all the 'y'.

However, I used 1:2 ratio when converting 'b' to 'y' and then checking it by putting boundary values like 'abb'. This make me safe of by one error.

Task 3

Task 3 – A:

To decrement the binary value, the machine uses a right to left borrow rule. The tape starts reading with from the left most bit and moves to the right to the 'B', then it moves one step to the left.

If the bit is a '0' it cannot be decremented directly, so the machine flips it to '1' which represents a borrow and steps to the left to borrow from the next higher bit.

If the current bit is 1, the borrow is satisfied and and machine flips to a '0' and stops borrowing. Once the borrow is done the the tape head moves back to the left start of the number and halts.

Task 3 – B:

(q0, '0') -> ('0', R, q0)

(q0, '1') -> ('1', R, q0)

(q0, 'B') -> ('B', L, q1)

(q1, '0') -> ('1', L, q1)

(q1, '1') -> ('0', L, q2)

(q2, '0') -> ('0', L, q2)

(q2, '1') -> ('1', L, q2)

(q2, 'B') -> ('B', R, q3)

Task 3 - C:

Starting Binary String B : 1001001

Decimal Value of B: 73

Target Operation: Decrement (-1)

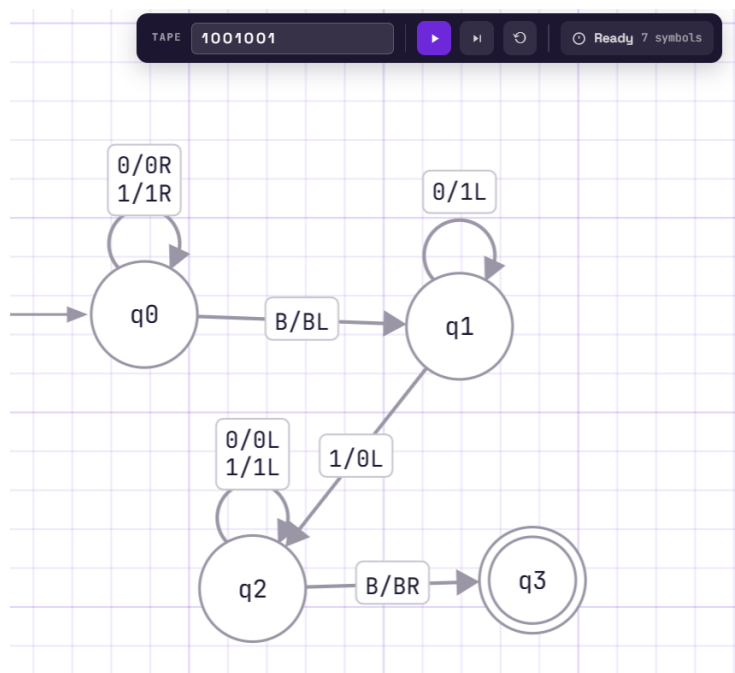
Expected Decimal Result: $73 - 1 = 72$

Expected Binary Result: 1001000

Final Tape Content from Simulator: 1001000

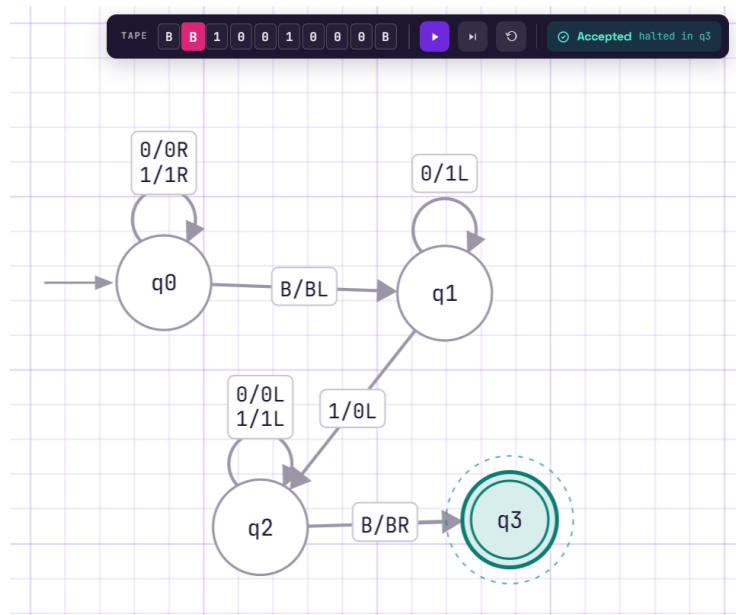
Before:

Location: Task3\png\q3_1001001_decrement_before.png



After:

Location: Task3\png\q3_1001001_decrement_after.png



Task3 - D:

Just changing the left most bit wont give us the correct result. If we take the example of an attempt. The $b = 1000$ this is the value 8 . if we just flip the left most bit it will be 0 instead of 7.

The correct method would be using the borrow rule – turning every 0 to 1 while moving forward, and flipping the first 1 and flipping it to a 0.

Task 4

Task4 – A:

Starting at the beginning, the machine reads the first letter (*a*, *b*, or *c*), erases it by writing a blank (*B*), and remembers which letter it saw by transitioning to a specific state branch (*q_saw_a*, *q_saw_b*, or *q_saw_c*). then the tape head scan all the way right, The tape head moves right past all remaining characters until it reaches the blank space (*B*) at the end of the string. Then to check the matching rightmost letter It steps one cell left to read the last letter.

If it matches the letter remembered by the state, the machine erases it (writes *B*) and sweeps all the way back left to start the process over.

If it is blank (*B*), it means the string was an odd length and the middle character was just erased—so the machine accepts.

If it does not match, the string isn't a palindrome, so the machine stops and rejects.

This left-and-right wiping loop continues. If *q0* reads a blank (*B*) right away, the whole string has been successfully matched in pairs (or was empty to begin with), so it transitions to *q_accept*.

Task 4 - B;

(*q0*, "abba") reads a writes B, moves R -> (*q_saw_a*, "bba")

(*q_saw_a*, "bba") reads b writes b, moves R -> (*q_saw_a*, "ba")

(*q_saw_a*, "ba") reads b writes b, moves R -> (*q_saw_a*, "a")

(*q_saw_a*, "a") reads a writes a, moves R -> (*q_saw_a*, "B")

(*q_saw_a*, "B") reads B writes B, moves L -> (*q_check_a*, "a")

(*q_check_a*, "a") reads a writes B, moves L -> (*q_rewind*, "bb")

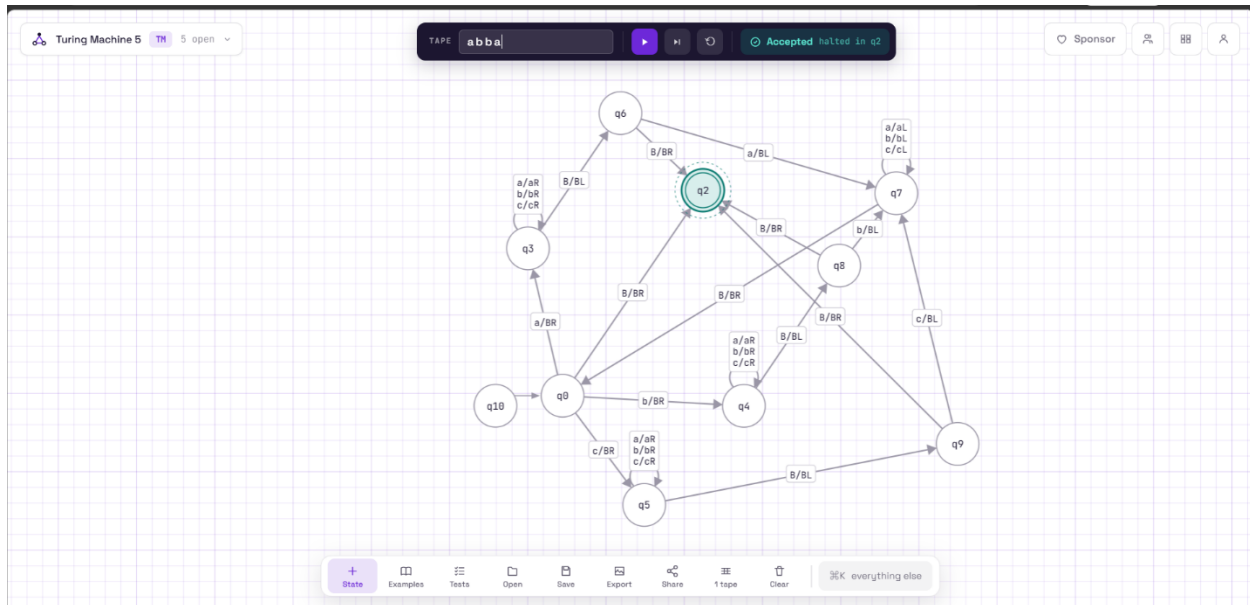
(*q_rewind*, "bb") reads b writes b, moves L -> (*q_rewind*, "b")

(q_rewind, "b") reads b writes b, moves L -> (q_rewind, "B")
(q_rewind, "B") reads B writes B, moves R -> (q0, "bb")
(q0, "bb") reads b writes B, moves R -> (q_saw_b, "b")
(q_saw_b, "b") reads b writes b, moves R -> (q_saw_b, "B")
(q_saw_b, "B") reads B writes B, moves L -> (q_check_b, "b")
(q_check_b, "b") reads b writes B, moves L -> (q_rewind, "B")
(q_rewind, "B") reads B writes B, moves R -> (q0, "B")
(q0, "B") reads B writes B, moves R -> (q_accept, "")

Task 4 - C:

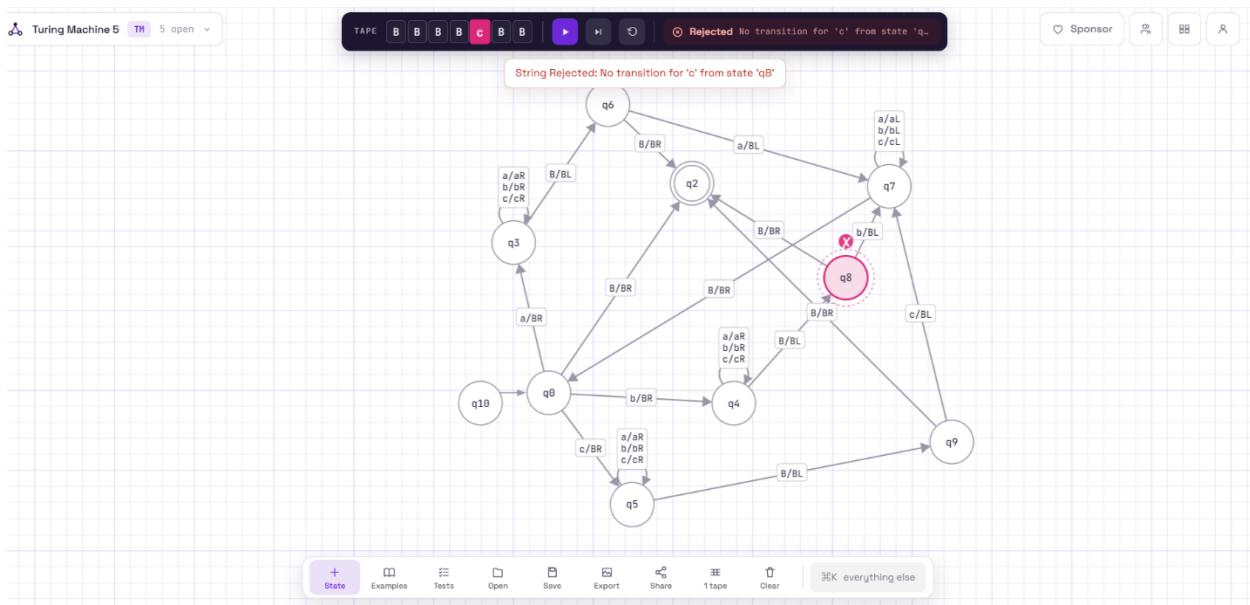
Valid Palindrome Input ("abba"): Accepted

Location: \Task4\PNG\q4_valid.png



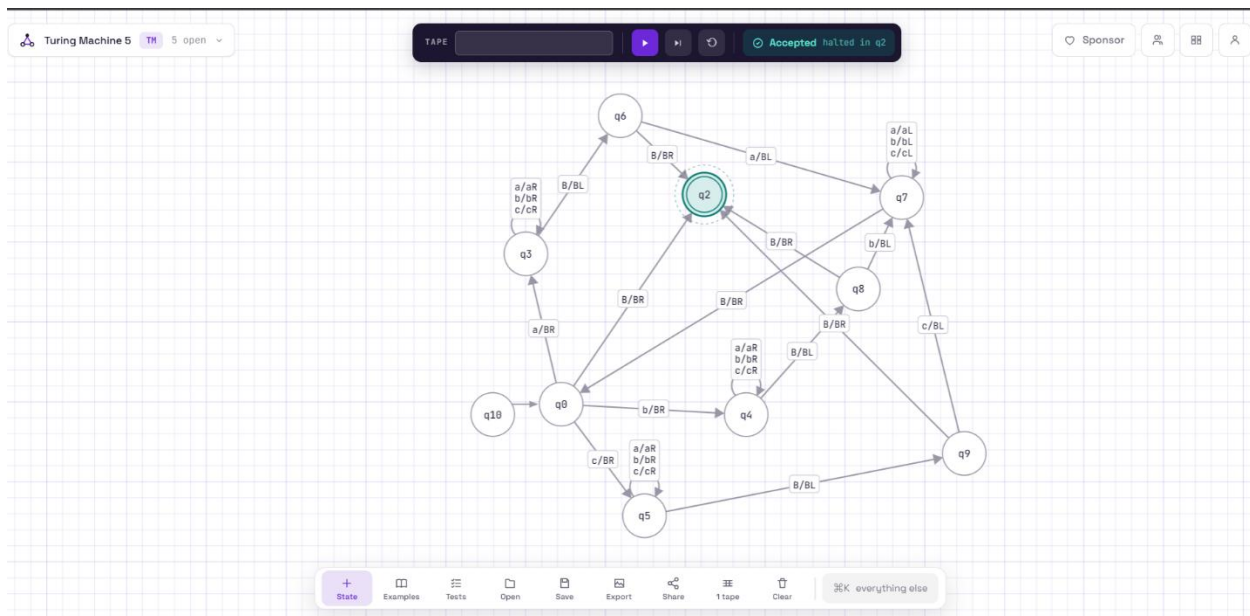
Invalid String Input ("abca"): Rejected

Location: \Task4\PNG\q4_Invalid.png



Valid Palindrome Input (" "): Accepted

Location: \Task4\PNG\q4_valid_empty.png



Task 4 - D:

A Turing machine cannot verify a palindrome in a single left to right pass because its internal memory is finite, meaning it cannot store an entire arbitrary sequence of characters at once. To compare the first symbol with the last symbol, the second with the second to last, and so on, the tape head is forced to physically move back and forth between both ends. A single pass would only allow reading the string once, leaving no way to cross-check the second half of the tape against what was seen in the first half.

Task 5

Task 5 – A:

To check if a string matches the language $w\#reverse(w)$, the machine compares bits from the outside inward—matching the first bit of w with the very last bit on the right side of $\#$.

Starting at the beginning of the tape, the machine reads the first bit 0 or 1, erases it by writing a blank B, and transitions to a state branch q_saw0 or q_saw1 to store that bit in memory. The head moves right across all remaining bits and the $\#$ symbol until it reaches the blank space B at the far right end of the tape. It steps one cell left to check the last bit on the right side. If it matches the remembered bit, the machine erases it (writes B) and prepares to rewind. If it does not match or if it hits $\#$ before finding a bit, the string is not a valid reversal, so the machine rejects.

The head sweeps left past $\#$ until it hits the starting blank boundary (B) on the left, then steps one cell right onto the next bit of w to repeat the process. Once every bit of w on the left has been erased, state $q0$ reads the $\#$ symbol. It erases $\#$ and steps right to confirm no extra leftover bits remain on the tape. If the tape is completely empty, it transitions to q_accept .

Task 5 – B:

$(q0, '0') \rightarrow ('B', R, q_saw0)$

$(q0, '1') \rightarrow ('B', R, q_saw1)$

$(q0, '\#') \rightarrow ('B', R, q_check_empty)$

$(q_saw0, '0'/'1'/'\#') \rightarrow (same, R, q_saw0)$

$(q_saw0, 'B') \rightarrow ('B', L, q_check0)$

$(q_check0, '0') \rightarrow ('B', L, q_rewind)$

$(q_saw1, '0'/'1'/'\#') \rightarrow (same, R, q_saw1)$

(q_saw1, 'B') -> ('B', L, q_check1)

(q_check1, '1') -> ('B', L, q_rewind)

(q_rewind, '0'/'1'/'#') -> (same, L, q_rewind)

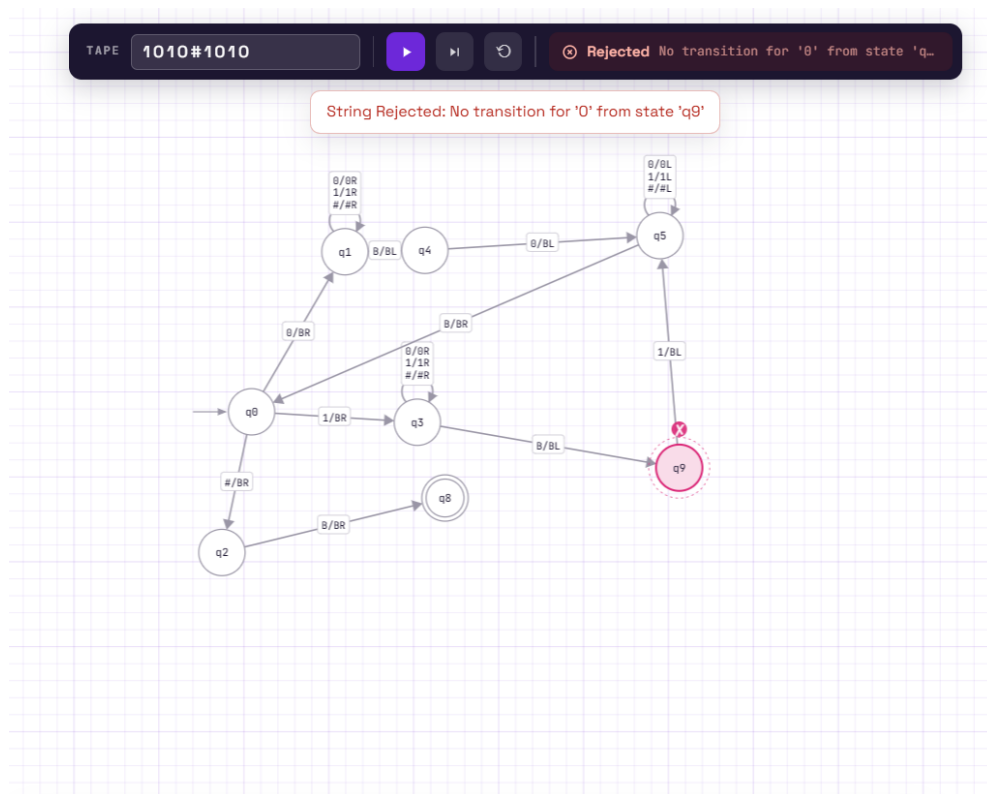
(q_rewind, 'B') -> ('B', R, q0)

(q_check_empty, 'B') -> ('B', R, q_accept)

Task 5 - C:

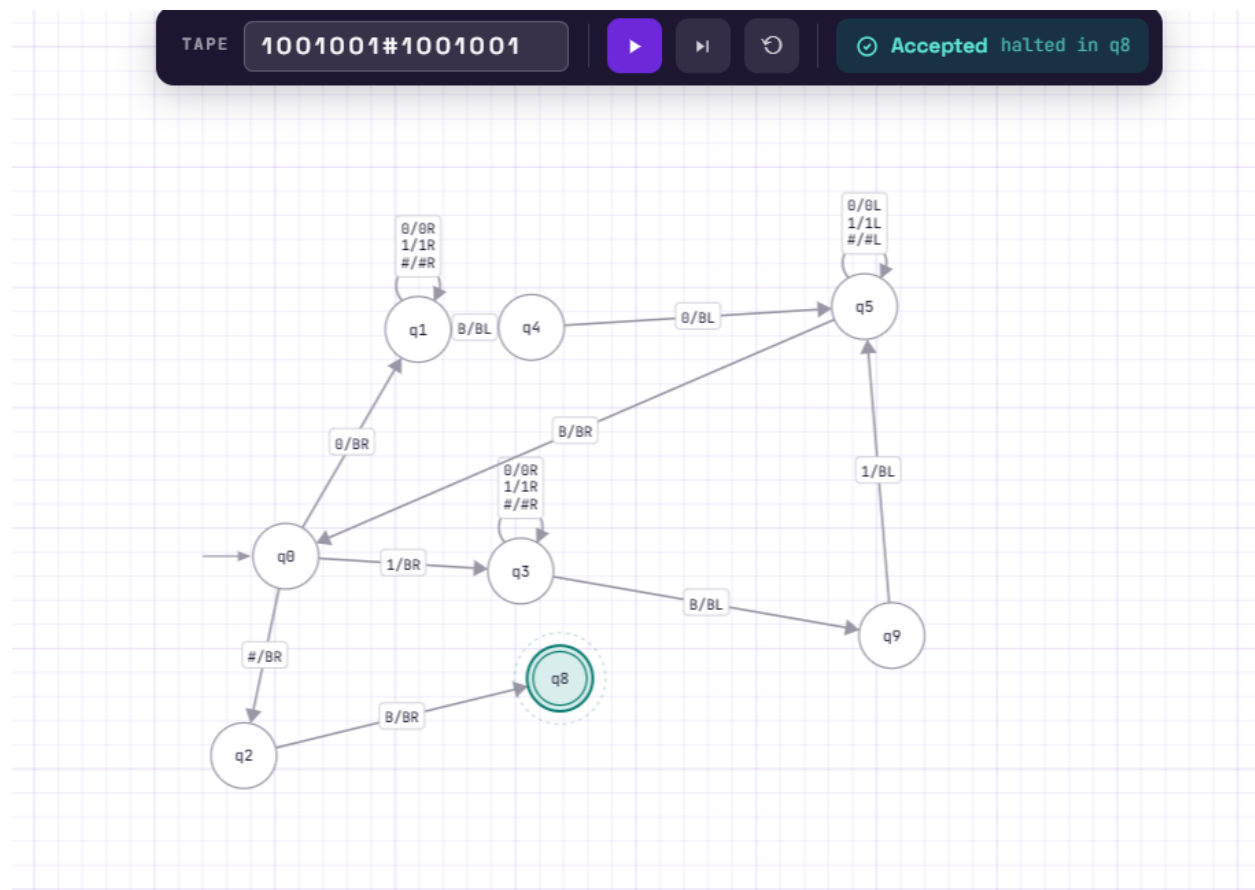
Valid Member String: 1010#1010 → Rejected

Location: \Task5\png\q5_Invalid.png



Invalid Non-Member String: 1001001#1001001 → Accepted

Location: Task5\png\q5_valid.png



Task 5 – D:

A Turing machine cannot recognize $w\#reverse(w)$ in a single left to right pass because its internal state control has finite memory and cannot store a long binary sequence w . To verify that the right side is the exact reverse of w , each bit on the left must be cross-checked against its mirror bit on the right. Because the tape head can only read one cell at a time, the head is forced to travel back and forth across the `#` to verify and mark each pair one by one.

Task 6

Task 6 - A:

For the input 11 – an even string.

(q_even, "11") → reads 1 → moves to q_odd

(q_odd, "1") → reads 1 → moves to q_even

(q_even, "empty") → reads empty → q_odd

(q_odd, "empty") → reads empty → q_reject

Task6 - B:

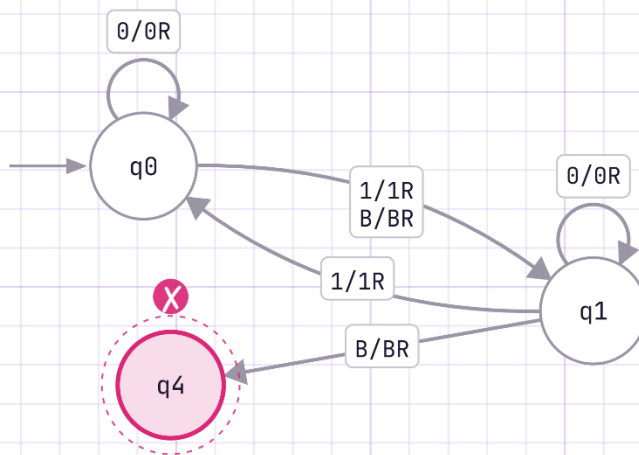
According to the trace table, when the even state reads empty on the tape (row 3) it should move to q_accept, not to q_odd. The trace show that when q_odd reads empty it moves to q_reject, however we should have transitioned from q_even to acceptance when it reads empty at q_even, the string would be even. so when empty string would be encountered after reading 1 and moving to q_even. Basically, when the tape reads empty in q_even then the string has ended in even number of 1's. So it should not go back to q_odd.

TASK 6 - C:

Broken: Located in task6/broken/

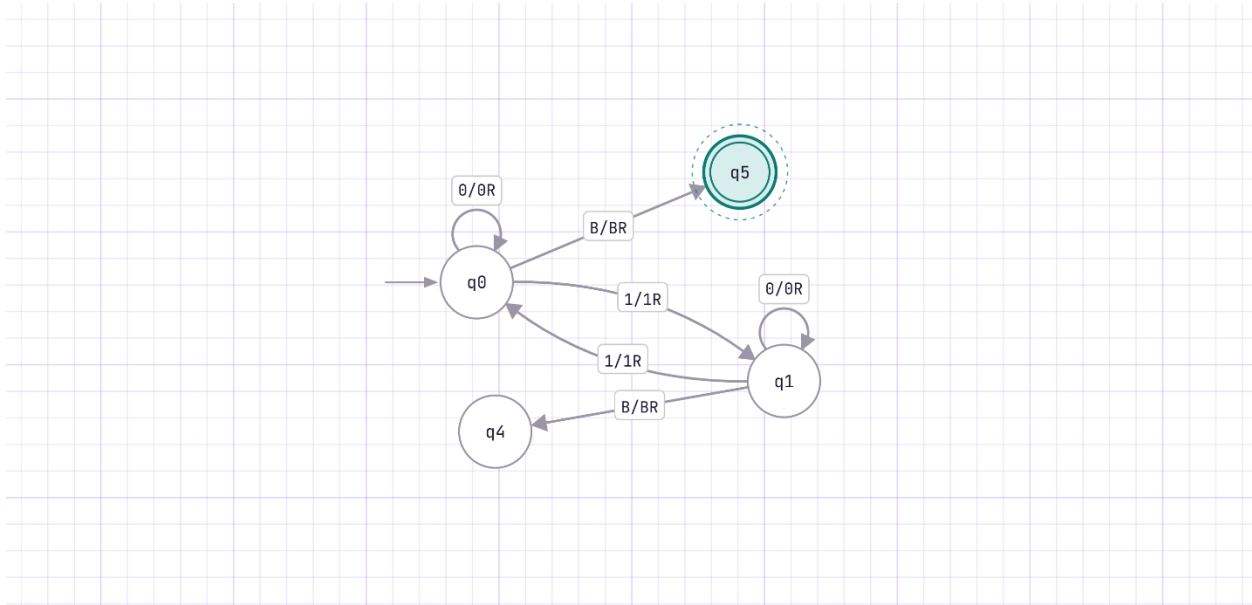
Screenshots: Task6\q6_broken_empty.png, Task6\q6_broken_11.png, Task6\q6_broken_1.png, Task6\q6_broken_111.png

(below only one image of png is attached to highlight the difference in the machines. The screenshot of the inputs is in the folder as there were 8 of them in total and would take up to much space)



Repaired : Located in task6/repair/

Screenshots: Task6\6_repair_empty.png, Task6\q6_repair_11.png,
Task6\q6_repair_1.png, Task6\q6_repair_111.png



Task 6 - D: The test strings "1" and "111" alone would not have revealed the bug because they are odd-parity inputs whose expected result is rejection—which happened to match what the broken machine produced. The broken machine did not have an acceptance state which means that it would produce right output on odd cases. So it is important to use both odd and even inputs, as the testing could be wrong when only focusing on a single or narrowed variety of inputs.